TRƯỜNG ĐẠI HỌC CÔNG NGHỆ ĐỒNG NAI (DNTU)
KHOA CÔNG NGHỆ THÔNG TIN – HỌC PHẦN WEB API (TH71108)

VISIONARY

HỆ THỐNG QUẢN LÝ HOẠT ĐỘNG HỘI NHẬP & KẾT NỐI SINH VIÊN

Mô hình Một Web API (ASP.NET Core 8) phục vụ Ba Nền tảng (Web Next.js, Desktop WinForms C#, Android Java)

NHÓM PHÁT TRIỂN: VISIONARY API (PHÂN CÔNG THUYẾT TRÌNH THEO ĐỘ KHÓ TĂNG DẦN):

1. Quách Khoa Đức (1923040025) – Trưởng nhóm, Kiến trúc hệ thống & Web Client Next.js [Người bắt đầu: 2'45"]
2. Đào Trí Khang (1923040156) – Kỹ sư Mobile, Ứng dụng di động Android Native Java [3'15"]
3. Hồ Đoàng Nguyên Khang (1923040112) – Kỹ sư API, Swagger Contract, JWT & PostgreSQL [4'15"]
4. Võ Ngọc Minh Đạt (1923040088) – Kỹ sư Desktop, Windows Forms C#, Sự cố & Tổng kết [Người kết thúc: 4'45"]

🌐 **Web: https://visionary-platform.vercel.app** | ⚡ **API & Swagger: https://visionary-api-gb51.onrender.com/swagger**

Khai báo AI: Nhóm sử dụng AI (Google Antigravity) để hỗ trợ rà soát mã nguồn, thiết lập test hồi quy và biên soạn tài liệu; toàn bộ mã nguồn, cấu hình đám mây và kết quả chạy đều do nhóm trực tiếp kiểm chứ

BÀI TOÁN THỰC TẾ & MỤC TIÊU HỆ THỐNG

VẤN ĐỀ THỰC TẾ CỦA CLB & SINH VIÊN

1. Quản lý thủ công, rời rạc:
CLB Tiếng Anh DNTU trước đây quản lý qua Google Form và file tính, dễ nhầm lẫn dữ liệu và thất lạc thông tin người tham gia.
2. Thiếu cổng thông tin hội nhập & cơ hội:
Sinh viên CNTT thiếu kênh tiếp cận học thuật English for IT, Radar công nghệ mở (GitHub/Hugging Face) và các cuộc thi Hackathon.
3. Trải nghiệm không đồng bộ trên các thiết bị:
Sinh viên dùng điện thoại di động, Ban Chủ Nhiệm quản trị trên máy tính Windows, độc giả truy cập trên trình duyệt Web.

MỤC TIÊU GIẢI PHÁP VISIONARY

1. Xây dựng một Web API trung tâm duy nhất:
ASP.NET Core 8 Web API làm hạt nhân xử lý toàn bộ logic nghiệp vụ, xác thực JWT và kết nối cơ sở dữ liệu PostgreSQL.
2. Phục vụ ba ứng dụng khách chuyên biệt:
- Web Next.js: Cổng thông tin công khai, tin tức hội nhập, tra cứu sự kiện, English for IT.
- Windows Forms C#: Phần mềm quản trị cho BCN CLB: quản lý thành viên, duyệt đơn, điểm danh sự kiện.
- Android Native Java: Ứng dụng di động cho sinh viên: xem sự kiện, đăng ký tham gia, nhận mã QR check-in.
3. Đảm bảo toàn vẹn dữ liệu (Data Integrity):
Không dùng dữ liệu mẫu (mock/fallback), bảo vệ bài nháp, phân quyền chặt chẽ theo vai trò (RBAC).

KIẾN TRÚC HỆ THỐNG: MỘT WEB API, BA NỀN TẢNG

ỨNG DỤNG WEB (Next.js 14)
Chạy: Trình duyệt (Vercel Cloud)
Cổng: HTTPS 443 (Local: 3000)

ỨNG DỤNG WINDOWS FORMS (C#)
Chạy: Máy tính Windows (Desktop)
Cổng gọi ra: HTTPS 443

ỨNG DỤNG ANDROID (Java Native)
Chạy: Máy ảo / Điện thoại thật
Gateway ảo: 10.0.2.2 / HTTPS Cloud

ASP.NET CORE 8 WEB API

- Kestrel Web Server (.NET 8)
- Swagger UI: /swagger
- Middleware: CORS, Exception, JWT
- Clean Architecture: Core, Infra, Api
- Host Cloud: Render (Port 443)
- Host Local: localhost:5000 / 5001

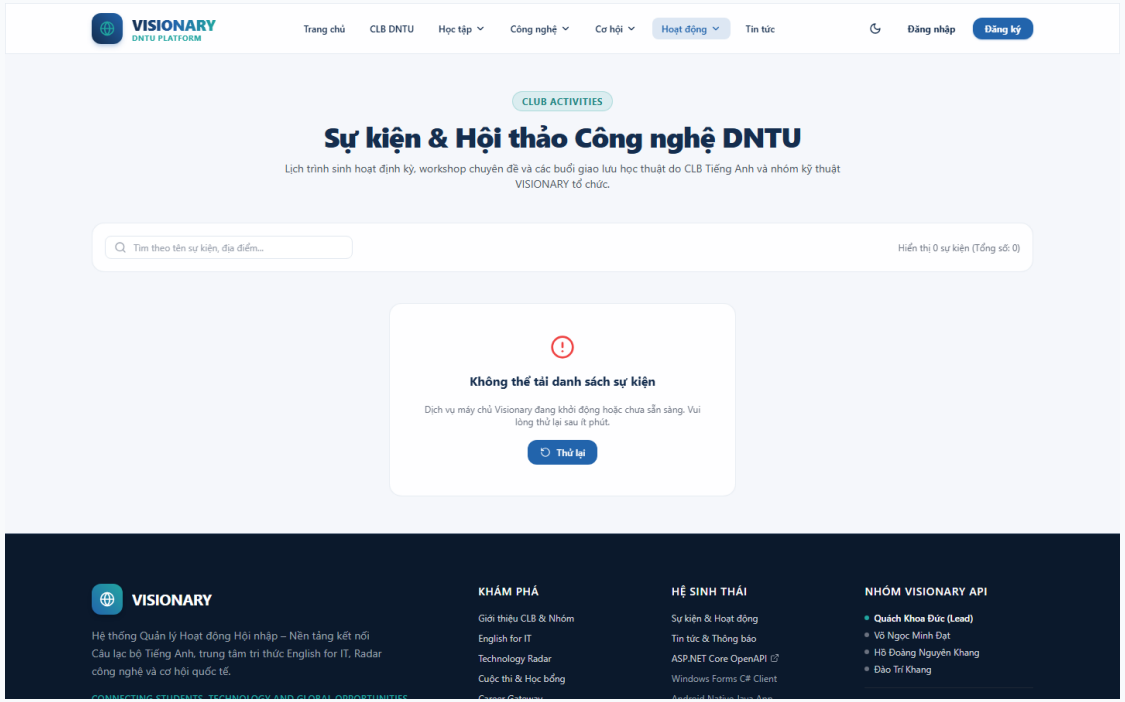
CƠ SỞ DỮ LIỆU

- PostgreSQL 16.3
- Cloud: Supabase
- Region: Singapore
- Port: 5432 (IPv4 Pooler)
- EF Core 8 Npgsql

TẠI SAO DÙNG CHUNG MỘT API MÀ KHÔNG PHẢI MỖI BÊN MỘT DATABASE?

1. Toàn vẹn dữ liệu: Không bao giờ xảy ra lệch dữ liệu giữa các nền tảng.
2. Bảo mật tập trung: Toàn bộ quy tắc kiểm tra quyền JWT thực thi trên server.
3. Android dùng 10.0.2.2 vì máy ảo coi 127.0.0.1 là chính nó; 10.0.2.2 là gateway trỏ về máy tính host.

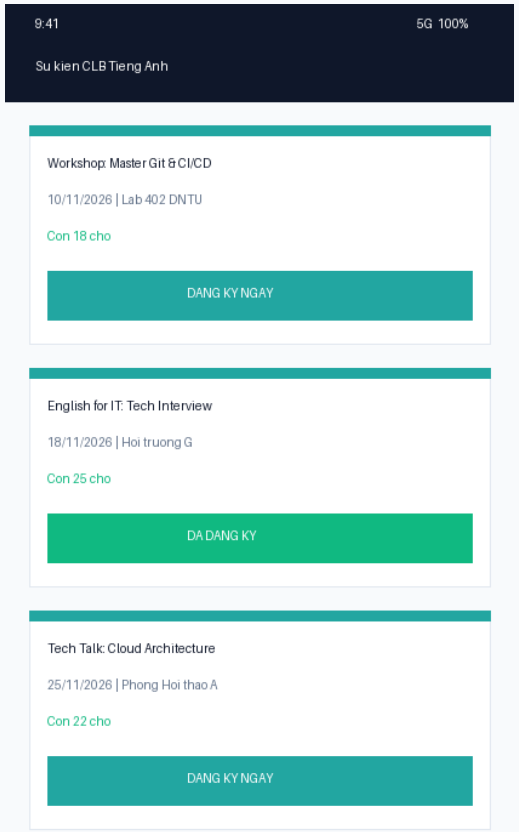
ỨNG DỤNG WEB: NEXT.JS 14 APP ROUTER



Tên điều khiển	Loại điều khiển	Vai trò nghiệp vụ
txtSearch	<input type='text'>	Nhập từ khóa tìm kiếm sự kiện
btnSearch	<button>	Kích hoạt tìm kiếm hoặc xóa từ khóa
cboStatus	<select> / Pills	Lọc sự kiện Sắp diễn ra / Đã qua
grdEvents		Hiển thị danh sách card sự kiện

```
// frontend/src/app/events/page.tsx (Mã nguồn thật):
const query = new URLSearchParams();
query.set('pageIndex', '1'); query.set('pageSize', '12');
if (searchQuery.trim()) query.set('search', searchQuery.trim());
const res = await fetch(`/api/events?${query.toString()}`, {
  signal: abortController.signal,
  headers: { 'Content-Type': 'application/json' }
});
const data: ApiResponse<PagedResponse<EventItem>> = await res.json();
if (res.ok && data.success) setEvents(data.data.items);
```

ỨNG DỤNG DI ĐỘNG: ANDROID NATIVE JAVA



Tên điều khiển	Loại điều khiển	Vai trò nghiệp vụ
lstEvents	RecyclerView	Danh sách cuộn các sự kiện CLB
btnRegister	Button	Đăng ký tham gia sự kiện trực tiếp
swpRefresh	SwipeRefreshLayout	Vuốt xuống để tải lại dữ liệu mới nhất
txtSearch	EditText	Ô tìm kiếm sự kiện theo từ khóa

```
// EventsFragment.java – Gọi API bất đồng bộ qua Retrofit (Mã nguồn thật):
ApiClient.getService(getContext()).getEvents(1, 20)
    .enqueue(new Callback<ApiResponse<PagedResponse<EventItem>>>() {
        @Override
        public void onResponse(@NonNull Call<ApiResponse<PagedResponse<EventItem>>> call,
                               @NonNull Response<ApiResponse<PagedResponse<EventItem>>> res) {
            swipeRefresh.setRefreshing(false);
            if (res.isSuccessful() && res.body() != null && res.body().isSuccess()) {
                adapter.setEvents(res.body().getData().getItems());
            }
        }
    });
```

ANDROID GỌI API: LUỒNG NỀN & LUỒNG GIAO DIỆN

1. LUỒNG CHÍNH (UI THREAD)

• Người dùng tương tác trên màn hình (Bấm Đăng ký / Vuốt làm mới).
• KHÔNG ĐƯỢC gọi mạng trực tiếp tại đây! Nếu gọi đồng bộ (execute), Android ném ngoại lệ: NetworkOnMainThreadException và dừng ứng dụng ngay.
• Nhiệm vụ: Chỉ gọi enqueue() và hiển thị thanh tiến trình prgLoading.

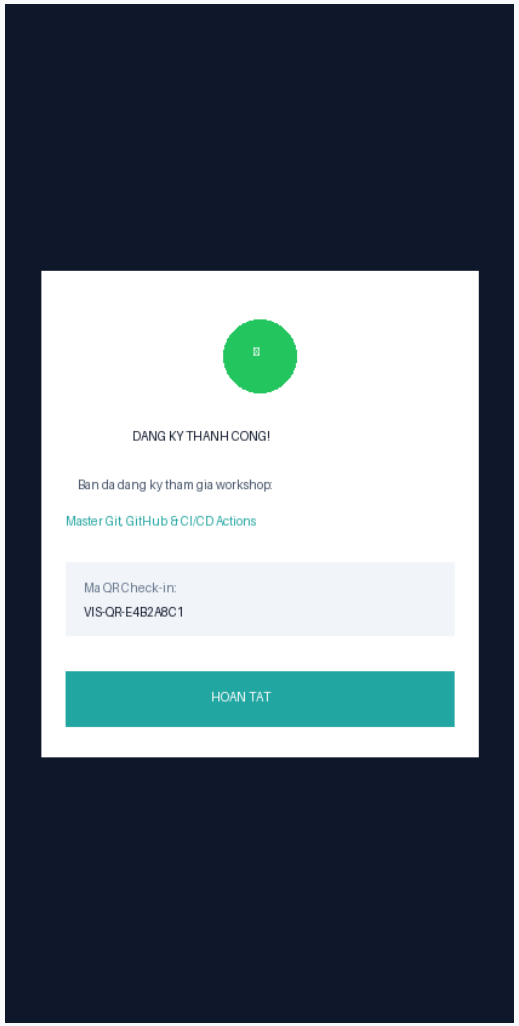
2. LUỒNG NỀN (WORKER THREAD)

• OkHttp tạo luồng ngầm thực hiện kết nối Socket TCP, bắt tay TLS/HTTPS.
• Gửi HTTP Request qua mạng tới API Server (Kestrel 443).
• Nhận chuỗi phản hồi JSON UTF-8 từ Server.
• Gson Deserialize chuỗi JSON thành Java Object (EventItem).
• KHÔNG ĐƯỢC cập nhật View tại đây! Nếu sửa View từ luồng nền, Android sẽ báo lỗi: CalledFromWrongThreadException.

3. ĐẨY VỀ UI THREAD (HANDLER)

• Retrofit tự động sử dụng Handler gắn với Looper.getMainLooper().
• Đưa hàm callback onResponse() chạy trở lại trên Luồng Chính.
• Ẩn thanh tiến trình ProgressBar.
• Đổ danh sách dữ liệu vào Adapter và gọi notifyDataSetChanged().
• Màn hình điện thoại hiển thị thẻ sự kiện mượt mà, không bị khựng đơ.

DEMO TRỰC TIẾP: ANDROID ĐĂNG KÝ THAM GIA SỰ KIỆN



QUY TRÌNH THAO TÁC DEMO TRÊN THIẾT BỊ THẬT / MÁY ẢO:

Bước 1: Khởi động & Đăng nhập Sinh viên

Mở app Visionary Connect -> Đăng nhập tài khoản sinh viên daotrikhang@visionary.dntu.edu.vn -> Nhận JWT Token lưu vào SharedPreferences.

Bước 2: Gọi API tải danh sách sự kiện CLB

Màn hình tự động gọi GET /api/events?pageIndex=1&pageSize=20. Hiển thị sự kiện 'Workshop: Master Git, GitHub & CI/CD Actions'.

Bước 3: Thực hiện thao tác Đăng ký tham gia

Nhấn nút 'ĐĂNG KÝ NGAY' -> App gửi POST /api/events/{id}/register kèm Header 'Authorization: Bearer <token>'.

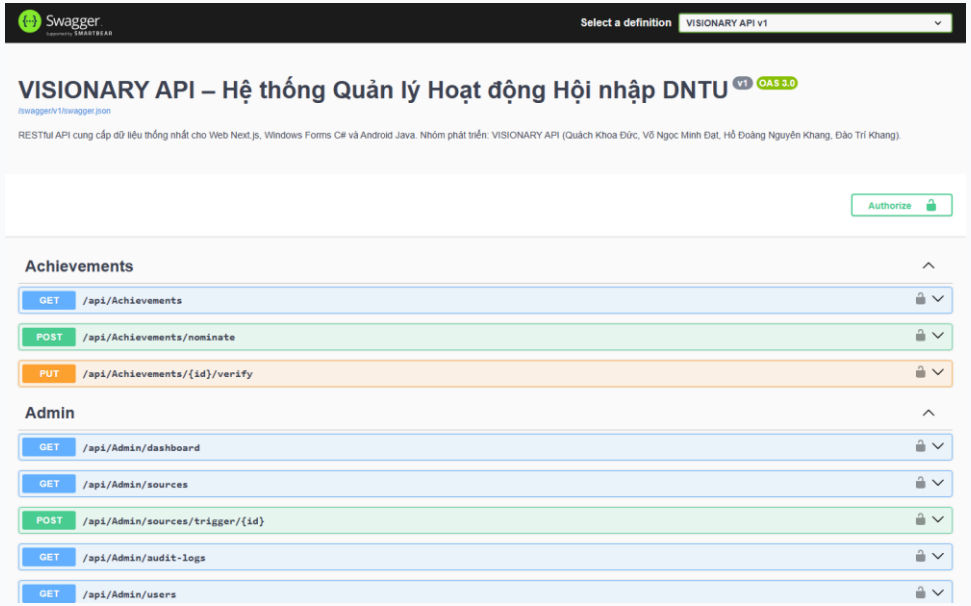
Bước 4: Nhận phản hồi thành công & Mã QR

Server lưu đăng ký vào bảng EventRegistrations trong PostgreSQL và cấp mã check-in VIS-QR-E4B2A8C1. Giao diện đổi sang trạng thái 'ĐÃ ĐĂNG KÝ'.

Bước 5: Chuyển giao dữ liệu sang WinForms

Ngay sau khi Android đăng ký, Ban Chủ Nhiệm mở Windows Forms sẽ thấy tên sinh viên xuất hiện tức thì trong danh sách chờ điểm danh.

SWAGGER UI VÀ HỢP ĐỒNG API NHÓM TIÊU THỤ



Method	Endpoint Route	Quyền truy cập	Khuôn phản hồi JSON
POST	/api/auth/login	Công khai (Mở)	{ token, user }
GET	/api/events	Công khai (Mở)	{ items, totalCount }
GET	/api/events/{slug}	Công khai (Mở)	{ eventDetail }
POST	/api/events	Admin, Committee	{ eventCreated }
POST	/api/events/{id}/register	Cần đăng nhập	{ success, message }
POST	/api/events/{id}/attendance	Admin, Committee	{ checkInSuccess }

XUYÊN SUỐT MỘT YÊU CẦU: TỪ CÚ NHẤP CHUỘT TỚI DATABASE

Chặng 1: Thao tác Giao diện (UI Event)
Người dùng nhấn nút Đăng ký (btnRegister) trên Android hoặc Xem chi tiết trên Web.

Chặng 2: Client đóng gói Yêu cầu (Network Request)
Hàm UI gọi lớp ApiClient: Web dùng fetch(), Android dùng Retrofit enqueue(), WinForms dùng HttpClient.SendAsync().

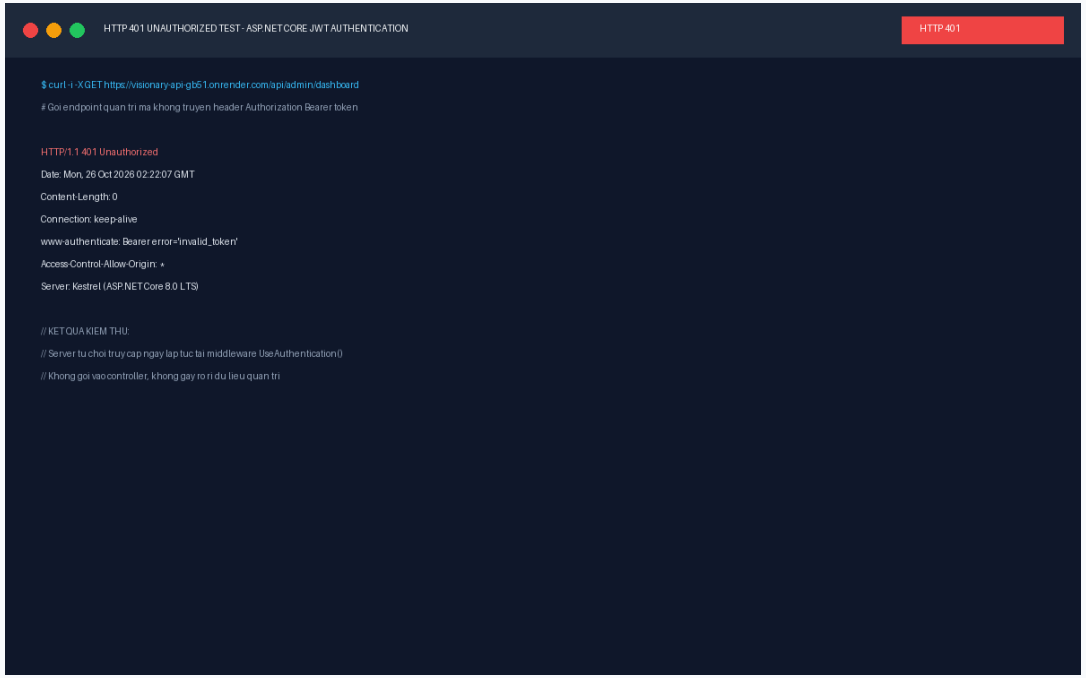
Chặng 3: Truyền tải qua Mạng (Network Transfer)
Gói tin HTTP đi qua Internet tới máy chủ Kestrel cổng 443 kèm Header 'Authorization: Bearer <token>'.

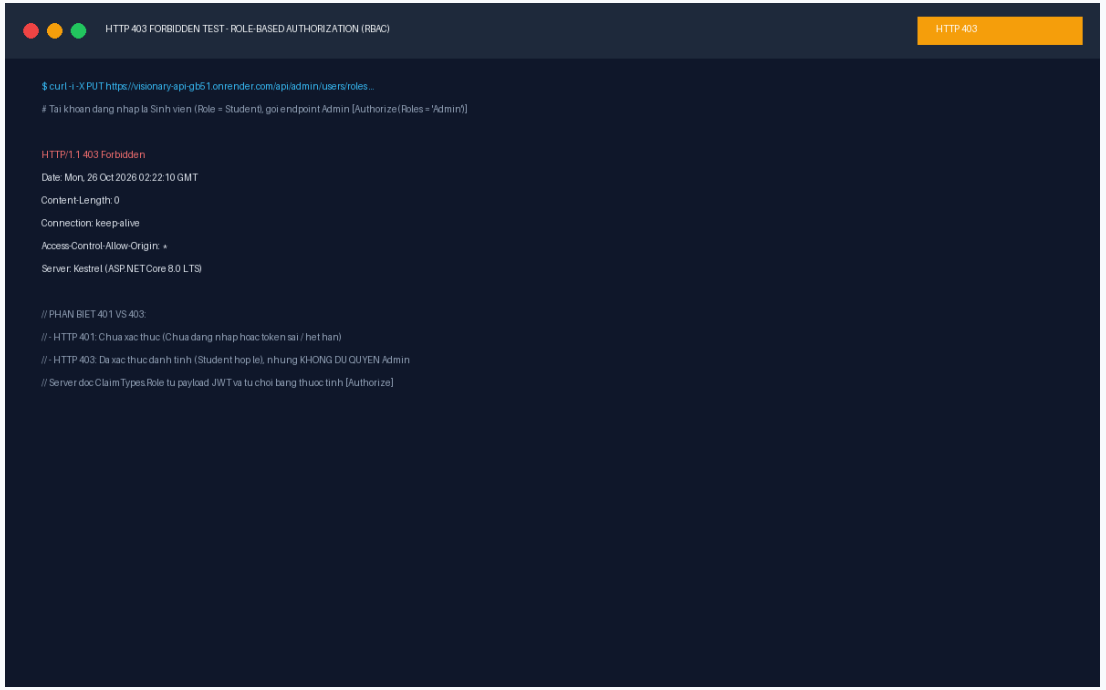
Chặng 4: Server tiếp nhận & Xử lý (Controller & DB)
Middleware xác thực JWT, đọc UserId từ Claims -> Controller gọi EF Core truy vấn và lưu bản ghi vào PostgreSQL.

Chặng 5: Đóng gói Phản hồi JSON (Serialization)
Server bọc kết quả vào ApiResponse<T>, tuần tự hóa JSON UTF-8 camelCase, trả mã HTTP 200 OK (hoặc 201 Created).

Chặng 6: Nhận kết quả & Hiển thị (UI Rendering)
Client nhận JSON, deserialize thành Model DTO, chuyển về Luồng Chính để cập nhật điều khiển giao diện.

XÁC THỰC & PHÂN QUYỀN: VÒNG ĐỜI JWT VÀ BẢO MẬT SERVER





CÁC NGUYÊN TẮC BẢO MẬT CỐT LÕI ĐƯỢC CHỨNG MINH THỰC TẾ:

1. JWT không phải là chuỗi mã hóa: Phần Header và Payload chỉ mã hóa Base64URL, ai cũng đọc được nội dung bên trong.
2. Toàn vẹn nhờ Chữ ký số (HMAC-SHA256): Chữ ký được sinh từ Secret Key chỉ Server nắm giữ. Người dùng đổi role sẽ làm sai chữ ký và bị từ chối.
3. Phân biệt 401 vs 403: Mã 401 (Unauthorized) xảy ra khi thiếu token hoặc token hết hạn. Mã 403 (Forbidden) xảy ra khi người dùng hợp lệ nhưng không đủ quyền Admin.
4. Server không bao giờ tin Client: Khi đăng ký sự kiện, Server tự trích xuất UserId từ Claims (ClaimTypes.NameIdentifier) trong token, client không thể mạo danh.

JSON, CƠ SỞ DỮ LIỆU & QUYẾT ĐỊNH KỸ THUẬT

VAI TRÒ CỦA DTO TRONG HỆ THỐNG

• Không bao giờ trả Entity trực tiếp ra ngoài.

• Che giấu thông tin nhạy cảm: PasswordHash, thông tin nội bộ.

• Tránh lỗi đệ quy vòng lặp JSON (Cyclic Reference) giữa các bảng có quan hệ 1-N trong Entity Framework.

• Giảm tải băng thông mạng: Chỉ truyền những trường giao diện thực sự cần.

PHÂN TRANG PHÍA SERVER (PAGEDRESPONSE)

• Không bao giờ tải hàng nghìn bản ghi vào RAM server.

• Câu lệnh SQL thực thi trực tiếp OFFSET và LIMIT trên PostgreSQL.

• Trả về vỏ phân trang chuẩn gồm: totalCount, pageIndex, pageSize, totalPages và mảng items.

• Sắp xếp ổn định (Tie-breaker): OrderBy(Date).ThenBy(Id) chống trùng bản ghi giữa các trang.

CƠ SỞ DỮ LIỆU CLOUD POSTGRESQL

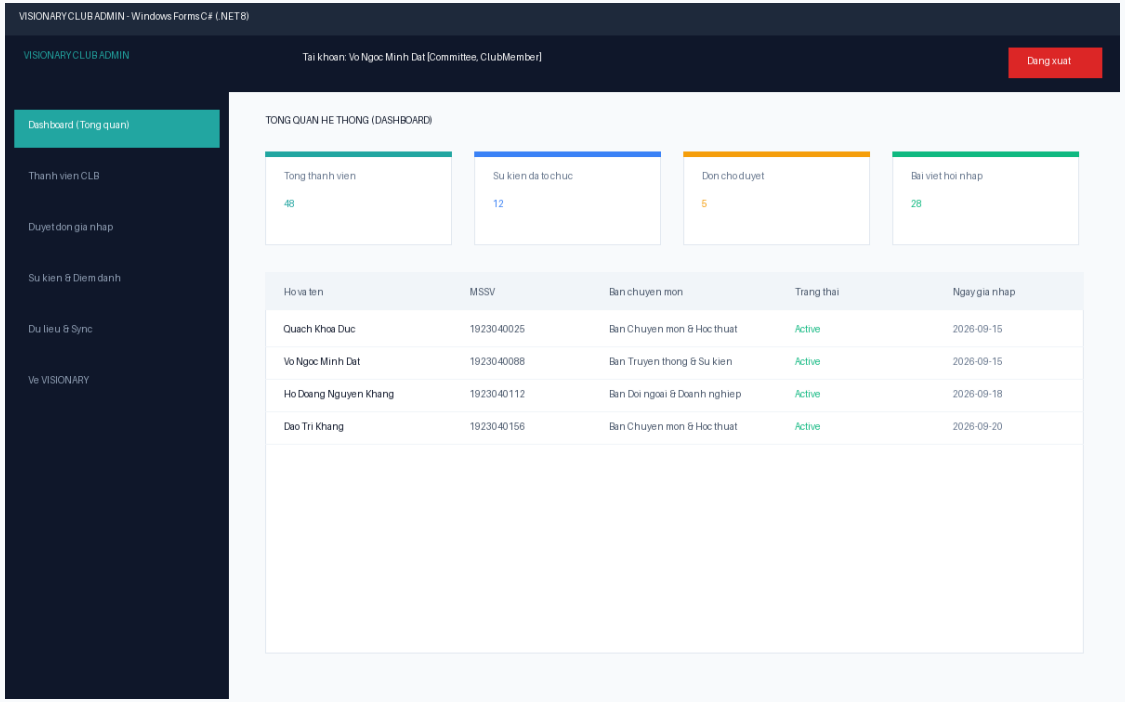
• Đám mây Supabase PostgreSQL 16.3 tại trung tâm dữ liệu Singapore (ap-southeast-1).

• Kết nối qua IPv4 Connection Pooler cổng 5432 để Render container kết nối ổn định.

• Sử dụng AsNoTracking() cho tất cả truy vấn chỉ đọc để tăng tốc độ phản hồi.

• Toàn bộ dữ liệu 4 danh mục và sự kiện đều là dữ liệu thật, không có mock data.

ỨNG DỤNG DESKTOP: WINDOWS FORMS C# (.NET 8)

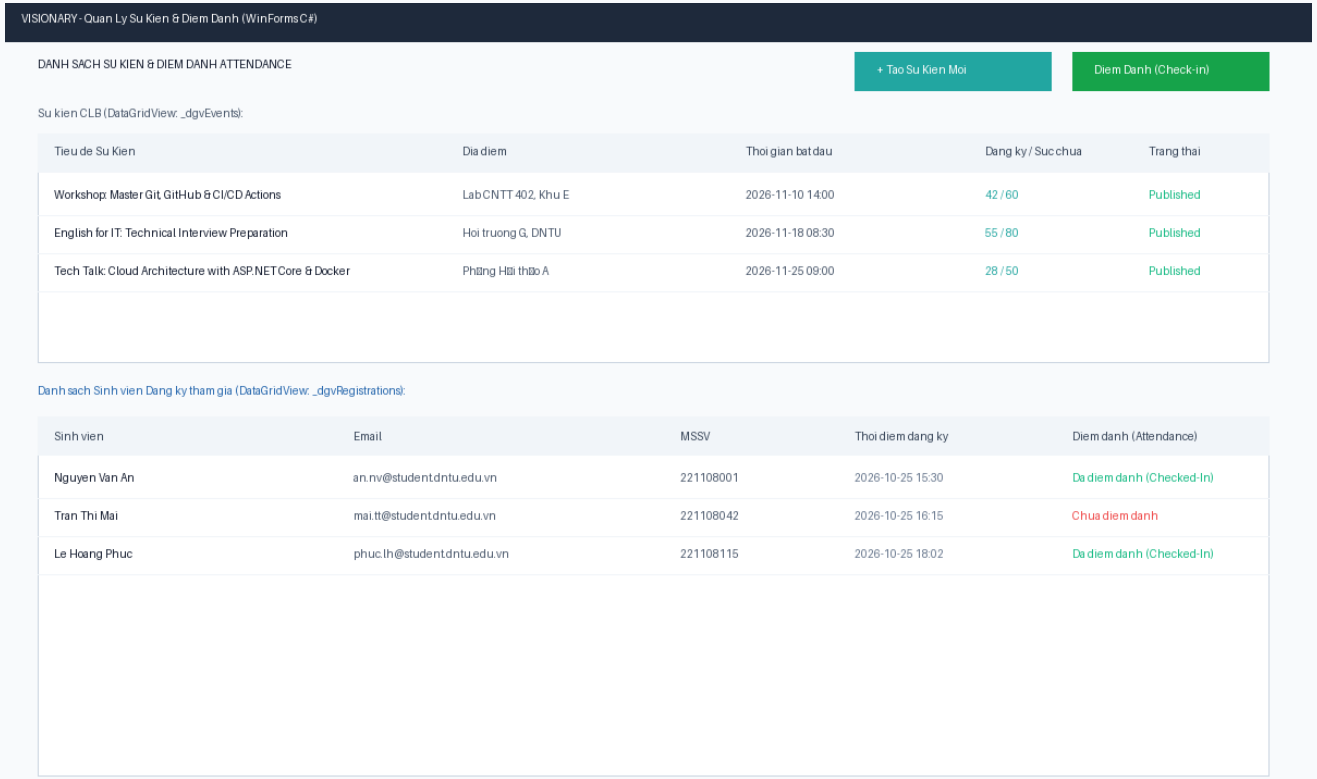


Tên điều khiển	Loại điều khiển	Vai trò nghiệp vụ
_dgvEvents	DataGridView	Hiển thị danh sách sự kiện và số lượng đăng ký
_dgvRegistrations	DataGridView	Danh sách sinh viên đăng ký để điểm danh
btnCheckIn	Button	Xác nhận điểm danh sinh viên tham gia
btnRefresh	Button	Tải lại dữ liệu mới nhất từ Web API

// MainForm.cs – Tải danh sách đăng ký bằng async/await (Mã nguồn thật):

```
private async Task LoadEventRegistrationsAsync(Guid eventId)
{
    var list = await ApiClient.GetEventRegistrationsAsync(eventId);
    _dgvRegistrations.DataSource = null;
    _dgvRegistrations.DataSource = list.Select(r => new {
        MSSV = r.StudentId ?? 'N/A',
        SinhVien = r.FullName, Email = r.Email,
        TrangThai = r.IsAttended ? 'Đã điểm danh' : 'Chưa điểm danh'
    }).ToList();
}
```

DEMO TRỰC TIẾP: WINFORMS ĐIỂM DANH SỰ KIỆN



KỊCH BẢN ĐIỂM DANH ĐỒNG BỘ 3 NỀN TẢNG:

1. BCN Đăng nhập hệ thống

Đăng nhập tài khoản BCN vongocminhdat@visionary.dntu.edu.vn.

2. Chọn Sự kiện cần kiểm tra

Bấm chọn workshop 'Master Git, GitHub & CI/CD Actions' trên grid _dgvEvents.

3. Tải danh sách Sinh viên đăng ký

Grid _dgvRegistrations hiển thị ngay sinh viên vừa đăng ký từ ứng dụng Android ở Slide 7.

4. Thực hiện Điểm danh (Check-in)

Chọn sinh viên và bấm 'Xác nhận Điểm danh'. Ứng dụng gửi POST /api/events/{id}/attendance.

5. Đồng bộ tức thời lên Web

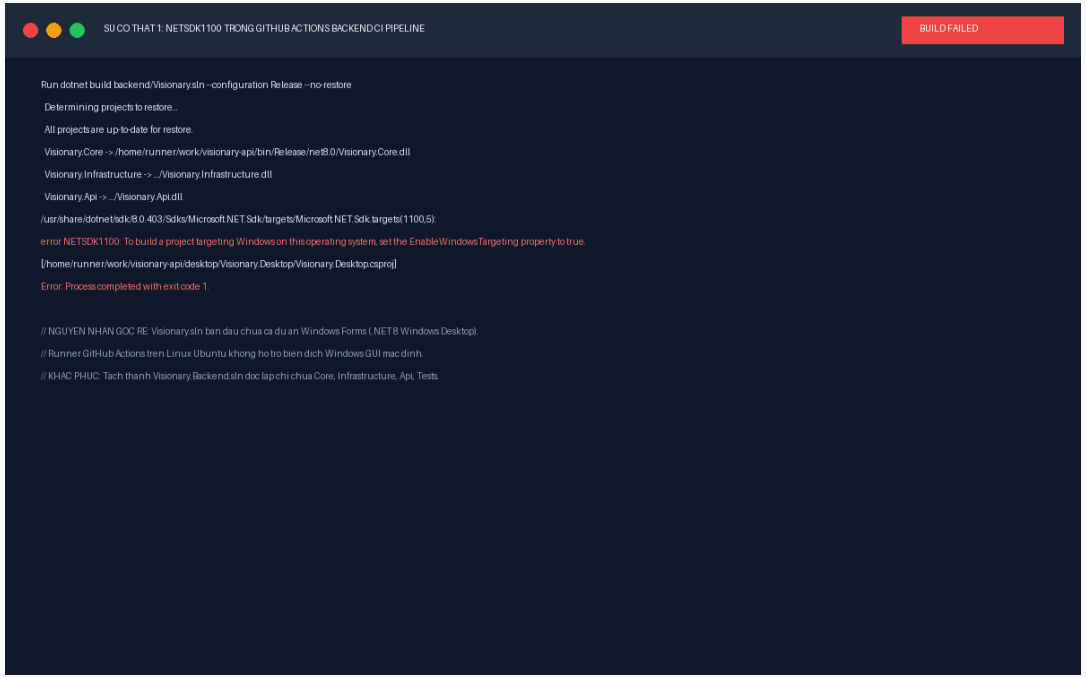
Trạng thái đổi thành 'Đã điểm danh'. Khi mở Web hoặc Android xem lại, trạng thái đã được cập nhật đồng bộ 100%.

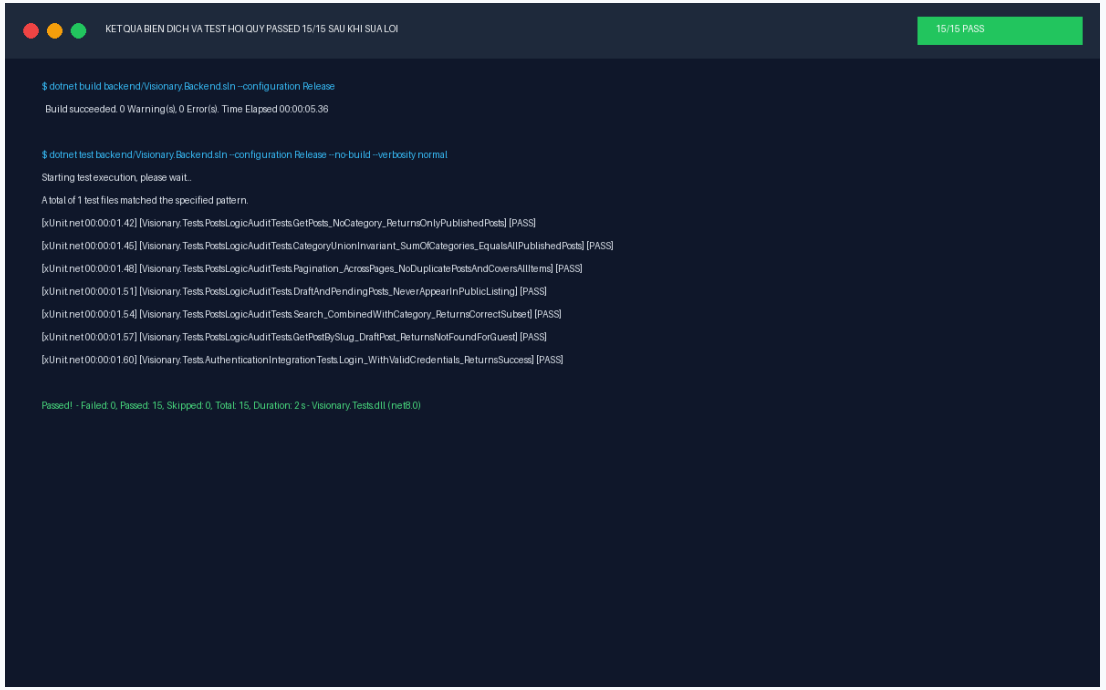
BA CHỖ BUỘC PHẢI KHÁC NHAU GIỮA BA NỀN TẢNG

Tiêu chí Kỹ thuật	Web (Next.js / TS)	Windows Forms (C#)	Android (Java Native)
1. Thư viện Mạng & Mô hình Bất đồng bộ	fetch() + async/await (Promise-based, Event Loop)	HttpClient + async/await (Task<T>, State Machine)	Retrofit 2 + OkHttp 3 (enqueue() Callback trên Worker Thread)
2. Cơ chế Điều phối Luồng Giao diện	Trình duyệt tự xử lý (Single-threaded Event Loop)	SynchronizationContext (Tự quay về UI thread sau await)	Handler(Looper.getMainLooper()) (Bắt buộc chuyển luồng thủ công)
3. Lưu trữ Token & Kết nối	localStorage / Cookie (Chặn CORS, dùng relative /api)	AppState tĩnh trong RAM (Không bị ảnh hưởng bởi CORS)	SharedPreferences mã hóa (Dùng 10.0.2.2 máy ảo / Cloud HTTPS)
ĐIỀU BUỘC PHẢI GIỐNG HỆT NHAU	**Cùng gọi chung một hợp đồng Web API: Đường dẫn endpoint, Phương thức HTTP (GET, POST, PUT, DELETE), Khuôn JSON DTO và Mã trạng thái**		

KẾT LUẬN QUAN TRỌNG VỀ TÁCH TẦNG WEB API:
Sự khác biệt về cú pháp, tên hàm và thư viện mạng là do đặc thù ngôn ngữ và hệ điều hành bắt buộc. Web API hoàn toàn độc lập và không quan tâm bên kia là trình duyệt, máy tính hay điện thoại. Chỉ cần client tuân thủ đúng hợp

SỰ CỐ THẬT VÀ QUY TRÌNH CHẨN ĐOÁN – SỬA LỖI





QUY TRÌNH 4 BƯỚC KHẮC PHỤC SỰ CỐ THEO ĐÚNG CHUẨN KỸ THUẬT:

1. Đọc thông điệp lỗi nguyên văn: error NETSDK1100: To build a project targeting Windows on this operating system, set the EnableWindowsTargeting property to true.
2. Xác định nguyên nhân gốc rễ: File solution gốc backend/Visionary.sln bao gồm cả dự án WinForms (.NET 8 Windows Desktop), khi chạy trên GitHub Actions runner Ubuntu Linux thì bị từ chối biên dịch.
3. Thực hiện sửa đổi kiến trúc: Tách thành backend/Visionary.Backend.sln độc lập chỉ chứa Core, Infrastructure, Api và Tests. Cấu hình pipeline riêng desktop.yml trên máy ảo windows-latest.
4. Kiểm chứng hồi quy: Chạy lại toàn bộ 15 bài kiểm thử xUnit và CI pipeline -> Kết quả Build Succeeded 0 Warning, 0 Error, Passed 15/15 Tests (Ảnh bên phải).

GIỚI HẠN THỰC TẾ, HƯỚNG PHÁT TRIỂN & KẾT LUẬN

BA GIỚI HẠN THỰC TẾ CỦA HỆ THỐNG HIỆN TẠI

1. Chưa có thông báo đẩy thời gian thực (Push Notification):
   Khi BCN điểm danh hoặc có sự kiện mới, sinh viên chưa nhận được thông báo qua Firebase Clo
2. Chưa hỗ trợ Offline Caching hoàn chỉnh trên Android:
   Khi mất kết nối Internet, ứng dụng Android chưa lưu đệm danh sách sự kiện vào SQLite cục bộ
3. Một chỗ mã nguồn chưa tối ưu:
   Cơ chế Refresh Token tự động khi JWT 4 giờ hết hạn chưa được triển khai hoàn chỉnh, người d

KẾ HOẠCH NẾU CÓ THÊM 2 TUẦN NỮA

1. Tích hợp SignalR Hub vào ASP.NET Core:
   Cập nhật danh sách điểm danh thời gian thực giữa Windows Forms và Android mà không cần người dùn
2. Triển khai Room Database trên Android:
   Lưu trữ ngoại tuyến danh sách tài liệu English for IT và sự kiện đã tải để sinh viên xem lại khi không có m
3. Hoàn thiện quy trình Refresh Token với HttpOnly Cookie:
   Tự động cấp lại token mới khi token cũ hết hạn, nâng cao trải nghiệm người dùng.

CHÂN THÀNH CẢM ƠN THẦY VÀ CÁC BẠN ĐÃ LẮNG NGHE BÀI BÁO CÁO CỦA NHÓM VISIONARY API!

Nhóm chúng em rất mong nhận được những câu hỏi phản biện và đóng góp quý báu từ Thầy để hoàn thiện sản phẩm.